Proyecto final Algoritmos

Modalidad: Parejas
Excepción: Debido a que hay 25 alumnos, se permitirá un único equipo de tres integrantes
Valor: 10 puntos
Lenguaje: C++
Herramientas que pueden utilizar: Visual Studio Code + GCC/g++, Code::Blocks

Descripción

El Proyecto Final tiene como propósito integrar los conocimientos adquiridos durante el curso de Algoritmos mediante el análisis, diseño, implementación, prueba y documentación de una solución computacional desarrollada en C++.

Cada equipo deberá construir un programa funcional que responda a uno de los casos asignados por el docente.

El propósito principal del proyecto no es desarrollar un sistema empresarial completo, sino demostrar dominio de los fundamentos de programación y pensamiento algorítmico estudiados durante el semestre.

El proyecto deberá evidenciar el proceso completo:

- Análisis del problema
- Diseño y Algoritmo
- Programación
- Pruebas
- Depuración
- Documentación

Por lo que deben diseñar e implementar una solución algorítmica estructurada en C++ que integre estructuras de control, modularización, estructuras de datos básicas y persistencia mediante archivos, aplicando buenas prácticas de programación, validación y prueba.

Objetivos

Al finalizar el proyecto, el estudiante deberá ser capaz de:

- Analizar un problema y establecer claramente sus entradas, procesos, salidas y restricciones.
- Diseñar una solución antes de programarla.
- Utilizar estructuras selectivas y repetitivas.
- Modularizar una solución mediante funciones.
- Utilizar arreglos cuando el problema lo requiera.
- Manipular cadenas de caracteres.
- Crear y utilizar registros mediante struct.
- Guardar y recuperar información mediante archivos.
- Validar entradas proporcionadas por el usuario.
- Implementar búsquedas y procesamiento de información.
- Diseñar casos de prueba.
- Identificar y corregir errores.
- Documentar y explicar técnicamente una solución.
- Trabajar colaborativamente y defender individualmente el código desarrollado.

Organización de equipos

Debido a que la sección cuenta con 25 estudiantes, se conformarán 11 parejas y se permitirá únicamente un equipo de 3 integrantes.

No se permitirán equipos adicionales de tres o más integrantes salvo un caso en especial o porque un integrante ya no va a continuar en el curso.

Cada integrante deberá participar activamente en: análisis, diseño, programación, pruebas, documentación, presentación.

El proyecto es colaborativo, pero la comprensión del código es individual.

Durante la defensa, el docente podrá solicitar a cualquier integrante que explique o modifique cualquier parte del programa.

Asignación de proyectos

Existen seis opciones, donde cada proyecto podrá ser asignado como máximo a dos equipos.

Las opciones son las siguientes:

- Sistema de control académico
- Sistema de inventario
- Sistema de biblioteca
- Sistema de ventas
- Sistema de gestión de empleados
- Sistema de reservas

Aunque dos equpos desarrollen la misma opción, cada solución deberá se propia, por lo tanto, no se aceptarán rogramas idénticos, estructuras idénticas de código o documentación compartida entre equipos.

Requesitos técnicos obligatorios

Independientemente del proyecto seleccionado, todos los programas deberán incluir como mínimo los siguientes elementos.

Menu principal

El programa deberá presentar un menu interactivo.

Ejemplo:

```
============================
MENÚ PRINCIPAL
============================
1. Registrar
2. Consultar
3. Listar
4. Modificar
5. Estadísticas
6. Guardar información
0. Salir
============================
Seleccione una opción:
```

El menú deberá repetirse hasta que el usuario seleccione la opción de salir.

Estructuras selectivas

El programa deberá utilizar correctamente el `if`, `if-else` y cuando corresponda `switch`

Las decisiones deberán responder a reglas reales del proyecto y no ser incluidas artificialmente únicamente para cumplir el requisito.

Estructuras repetitivas

El programa deberá utilizar al menos dos de las siguientes estructuras:

`while`

`do-while`

`for`

Deberán utilizarse para resolver necesidades reales como:

- menús
- validaciones
- recorridos
- búsquedas
- cálculos
- procesamiento de registros

Funciones

El programa deberá estar modularizado y no se permitirá desarrollar toda la solución dentro de int main(). Deberán existir como mínimo cinco funciones propias.

Ejemplo:

```
void registrarProducto();

void mostrarProductos();

int buscarProducto();

double calcularTotal();

void guardarArchivo();
```

El nombre de cada función deberá describir claramente su responsabilidad.

Estructuras de datos

Registros

El programa deberá utilizar al menos una estructura `struct`

Ejemplo:

```
struct Producto {
    int codigo;
    string nombre;
    double precio;
    int existencia;
};
```

Colecciones

Los registros deberán almacenarse utilizando alguna estructura estudiada durante el curso.

Por ejemplo: `Producto productos[100];`

No es obligatorio utilizar STL avanzada, sino que, el objetivo es demostrar dominio de los conceptos estudiados.

Cadenas

El programa deberá manejar información textual utilizando `string`

Ejemplos:

- nombres
- descripciones
- categorías
- autores
- clientes
- empleados

Cuando corresponda deberán utilizarse operaciones básicas de comparación, búsqueda o concatenación.

Archivos

El proyecto deberá permitir conservar información después de cerrar el programa, para esto se deberá utilizar manejo de archivos en C++ mediante fstream, ifstream o ofstream

Como mínimo, el programa deberá guardar registros y recuperar registros previamente almacenados. No se utilizarán bases de datos.

Validaciones

El sistema deberá validar las entradas importantes.

Ejemplos:

- cantidades negativas;
- opciones inexistentes;
- notas fuera de rango;
- registros duplicados;
- códigos inexistentes;
- campos vacíos;
- operaciones inválidas.

Ejemplo:

```
while (nota < 0 || nota > 100) {
    cout << "Nota invalida. Ingrese nuevamente: ";
    cin >> nota;
}
```

Búsqueda

Todo proyecto deberá implementar al menos una búsqueda.

Por ejemplo:

- buscar estudiante;
- buscar producto;
- buscar libro;
- buscar empleado;

- buscar venta;
- buscar reserva.

La búsqueda deberá realizarse utilizando algún identificador único.

Ejemplo:

- Código
- Carné
- ID
- Número de reserva

Reportes o estadísticas

El sistema deberá generar al menos dos cálculos o reportes derivados.

Ejemplos:

- promedio
- cantidad de registros
- mayor
- menor
- total acumulado
- cantidad aprobada
- inventario disponible
- ventas totales
- libros prestados

El objetivo es demostrar que el programa no solamente almacena información, sino que también la procesa.

Opciones de proyectos

1. Sistema de Control Académico

Una institución educativa necesita un programa que permita llevar un control básico de estudiantes y notas.

Datos mínimos

Cada estudiante deberá tener:

- carné;
- nombre;
- carrera;
- nota 1;
- nota 2;
- nota 3;
- promedio.

Funcionalidades mínimas

1. Registrar estudiante.
2. Validar notas entre 0 y 100.
3. Calcular promedio.
4. Determinar si el estudiante está aprobado o reprobado.
5. Buscar estudiante por carné.
6. Mostrar todos los estudiantes.
7. Mostrar promedio general del grupo.
8. Mostrar estudiante con mayor promedio.
9. Mostrar cantidad de aprobados.
10. Mostrar cantidad de reprobados.
11. Guardar información.
12. Recuperar información desde archivo.

2. Sistema de Inventario

Una pequeña empresa necesita controlar los productos disponibles en su inventario

Datos minimos

Cada producto debe contener:

- Código

- Nombre
- Categoría
- Precio
- Existencia

Funcionalidades

1. Registrar productos
2. Evitar códigos duplicados
3. Buscar producto por código
4. Mostrar inventario completo
5. Registrar entrada de unidades
6. Registrar salida de unidades
7. Evitar existencias negativas
8. Mostrar productos con existencia baja
9. Calcular valor total del inventario
10. Mostrar producto con mayor existencia
11. Guardar información
12. Recuperarla desde archivo

3. Sistema de Biblioteca

Una biblioteca necesita controlar sus libros y préstamos básicos.

Datos mínimos

Cada libro deberá contener:

- código;
- título;
- autor;
- categoría;
- estado.

En cuanto al estado, tendrá dos valores unicamente: Disponible y Prestado.

Funcionalidades

1. Registrar libros.
2. Buscar libro por código.
3. Buscar libros por título.
4. Mostrar todos los libros.
5. Prestar un libro.

6. Evitar prestar un libro que ya esté prestado.
7. Registrar devolución.
8. Mostrar libros disponibles.
9. Mostrar libros prestados.
10. Mostrar cantidad de libros por estado.
11. Guardar información.
12. Recuperar información.

4. Sistema de control de ventas

Una tienda necesita registrar sus ventas y generar estadísticas básicas.

Datos mínimos

Cada venta deberá contener:

- número de venta
- nombre del cliente
- producto
- cantidad
- precio unitario
- subtotal
- descuento
- total

Funcionalidades

1. Registrar venta.
2. Validar cantidad y precio.
3. Calcular subtotal.
4. Aplicar reglas de descuento.
5. Calcular total.
6. Buscar una venta.
7. Mostrar todas las ventas.
8. Calcular ventas acumuladas.
9. Mostrar venta de mayor valor.
10. Calcular promedio de ventas.
11. Guardar información.
12. Recuperar información.

5. Sistema de Gestión de Empleados

Una empresa necesita administrar información básica sobre sus trabajadores.

Datos mínimos

Cada empleado deberá contener:

- código;
- nombre;
- puesto;
- horas trabajadas;
- salario por hora;
- salario calculado.

Funcionalidades

1. Registrar empleado.
2. Evitar códigos repetidos.
3. Calcular salario.
4. Buscar empleado.
5. Mostrar empleados.
6. Mostrar empleado con mayor salario.
7. Mostrar salario promedio.
8. Calcular total de planilla.
9. Clasificar empleados según algún criterio definido.
10. Guardar información.
11. Recuperar información.

6. Sistema de Reservas

Una empresa de servicios necesita administrar reservaciones de sus clientes.

Puede contextualizarse como:

- hotel
- restaurante
- salón
- clínica
- cancha deportiva

Datos mínimos

Cada reserva deberá contener:

- código
- nombre del cliente
- fecha
- tipo de servicio
- cantidad de personas
- costo
- estado

Estados sugeridos: Activa, Cancelada, Finalizada

Funcionalidades

1. Crear reserva.
2. Buscar reserva.
3. Mostrar reservas.
4. Modificar estado.
5. Validar cantidad de personas.
6. Calcular costo.
7. Mostrar reservas activas.
8. Mostrar reservas canceladas.
9. Calcular ingresos estimados.
10. Mostrar total de reservas.
11. Guardar información.
12. Recuperar información.

Fases del proyecto

El proyecto no deberá desarrollarse únicamente durante la última semana.Se trabajará mediante entregas parciales.

Fase 1 – Análisis

Cada equipo deberá entregar:

- Nombre del proyecto
- Integrantes;
- Descripción del problema;
- Objetivo;

- Entradas;
- Procesos;
- Salidas;
- Reglas de negocio;
- Lista preliminar de funcionalidades.

Como entregable para esta fase será un Documento PDF.

Fase 2 – Diseño

Entregar:

- Diseño del menú
- Definición de struct
- Variables principales
- Funciones propuestas
- Pseudocódigo de las funcionalidades principales
- Casos de prueba iniciales

Fase 3 – Prototipo

El programa deberá permitir como mínimo:

- Registrar
- Mostrar
- Buscar
- Utilizar menú
- Utilizar funciones
- Utilizar ciclos

El programa deberá compilar y ejecutarse.

Fase 4 — Proyecto final

El sistema deberá contener todos los requerimientos establecidos. Será presentado en clase.

Documento PDF de entrega

Debe contener:

1. Portada.
2. Integrantes.
3. Descripción del problema.
4. Objetivo.
5. Análisis Entrada–Proceso–Salida.
6. Reglas de negocio.
7. Descripción de estructuras de datos.
8. Descripción de funciones.
9. Capturas del programa.
10. Casos de prueba.
11. Resultados obtenidos.
12. Problemas encontrados y cómo fueron solucionados.
13. Conclusiones individuales.
14. Declaración de uso de Inteligencia Artificial.

Presentación

Cada equipo realizará una demostración de aproximadamente 8 a 10 minutos.

Primera parte — Presentación del equipo

Aproximadamente 5 minutos.

Deberán mostrar:

- Problema
- Solución diseñada
- Funcionalidades principales
- Ejecución del programa

Segunda parte — Defensa

Aproximadamente 3 a 5 minutos, donde voy a seleccionar a cualquier integrante y preguntar:

- ¿Qué hace esta función?
- ¿Por qué utilizaron este ciclo?
- ¿Por qué eligieron esta estructura?

- ¿Qué ocurre con determinada entrada?
- ¿Dónde se guarda la información?
- ¿Qué modificaría para implementar una nueva regla?

Tambien voy a solicitar una modificación pequeña durante la defensa.

Ejemplo:

Cambie la condición de aprobación de 61 a 65 o agregue una validación para impedir valores negativos. El objetivo de la defensa es comprobar comprensión individual.